

Inteligência Artificial

Trabalho Final: Aplicação de Lógica Fuzzy na Classificação de Anomalias em Testes de Robustez de APIs REST

Levi Gomes¹, Luiz Garcia¹, Nata Cezer Bordignon¹, Renan Balestrin Bez¹

¹Universidade Regional Integrada do Alto Uruguai e das Missões – Campus Erechim
Erechim – RS – Brazil

Resumo. A classificação de anomalias em testes de APIs REST enfrenta o desafio persistente do oráculo de teste, especificamente na distinção entre falhas de implementação e comportamentos esperados sob condições de estresse. Este trabalho apresenta a implementação de um Sistema de Inferência Fuzzy (FIS) integrado a uma ferramenta de fuzzing guiada por contrato para automatizar essa triagem de forma inteligente. O sistema utiliza três variáveis difusas — desvio de status code, tempo de resposta e tamanho do payload — para calcular um score de criticidade contínuo entre 0.0 e 1.0. Implementada em Python com a biblioteca scikit-fuzzy, a solução foi validada contra a aplicação vulnerável crAPI. Os resultados revelaram 207 anomalias, das quais apenas 42 foram classificadas como de alta severidade pelo motor fuzzy, demonstrando maior precisão e granularidade na priorização de falhas em relação a abordagens binárias tradicionais. A metodologia mostra-se promissora para integração em pipelines de CI/CD, otimizando o ciclo de desenvolvimento de software robusto.

Abstract. Anomaly classification in REST API testing faces the persistent challenge of the test oracle, specifically in distinguishing between implementation failures and expected behaviors under stress conditions. This paper presents the implementation of a Fuzzy Inference System (FIS) integrated with a contract-guided fuzzing tool to automate this triage intelligently. The system employs three fuzzy variables — status code deviation, response time, and payload size — to calculate a continuous criticality score between 0.0 and 1.0. Implemented in Python using the scikit-fuzzy library, the solution was validated against the crAPI vulnerable application. Results revealed 207 anomalies, of which only 42 were classified as high severity by the fuzzy engine, demonstrating greater precision and granularity in prioritizing failures compared to traditional binary approaches. The methodology proves promising for integration into CI/CD pipelines, optimizing the development cycle of robust software.

1. Introdução

As APIs REST consolidaram-se como o padrão dominante para a integração de sistemas modernos, servindo de base para microsserviços, aplicações móveis e ecossistemas B2B. Para garantir a confiabilidade dessas interfaces, ferramentas de *fuzzing* guiado por contrato [Meyer 1992], como o protocolo OpenAPI [OpenAPI Initiative 2020], automatizam a descoberta de falhas de robustez ao explorar sistematicamente o espaço de entrada das aplicações. Entretanto, um desafio central nesse processo é a definição do **oráculo de**

teste: a capacidade de determinar se uma resposta inesperada do servidor constitui uma falha real ou um comportamento aceitável sob estresse.

Abordagens **binárias** tradicionais, baseadas puramente em lógica booleana (passa/falha), mostram-se insuficientes para lidar com fenômenos contínuos e incertos, como variações na latência de rede, *payloads* limítrofes e erros intermitentes. Tais métodos frequentemente geram uma alta taxa de falsos positivos ao rotular qualquer desvio do contrato como uma falha crítica. Neste contexto, a **Lógica Fuzzy**, introduzida por [Zadeh 1965], oferece um arcabouço matemático robusto para lidar com a incerteza e a gradação, permitindo uma classificação mais refinada e realista das anomalias detectadas.

1.1. Problema de Pesquisa

Um contrato OpenAPI especifica formalmente os códigos de *status* e os esquemas de resposta esperados para cada *endpoint*. Durante a execução de testes de *fuzzing*, a API pode retornar respostas que desviam do contrato de formas sutis, como códigos 400 não documentados, respostas 200 que violam o esquema estrutural, ou erros 500 intermitentes acompanhados de tempos de resposta anômalos. Classificadores booleanos rígidos tendem a agrupar todas essas ocorrências sob um mesmo rótulo de “falha”, o que dificulta a priorização de correções por parte das equipes de desenvolvimento.

O problema reside, portanto, em **como ranquear essas anomalias por sua criticidade real**, distinguindo de forma inteligente entre um erro de validação esperado e uma falha grave, como vazamento de dados ou colapso do servidor.

1.2. Pergunta de Pesquisa

A partir do problema exposto, formula-se a seguinte questão: “*O uso de um Sistema de Inferência Fuzzy (FIS), combinando as variáveis de desvio de código de status, tempo de resposta e variação do tamanho do payload, permite classificar anomalias de robustez em APIs REST com maior precisão e granularidade do que abordagens puramente booleanas?*”

1.3. Objetivos

Objetivo Geral

Implementar um **Sistema de Inferência Fuzzy** para classificar as respostas de APIs REST em níveis de criticidade (*LOW*, *MEDIUM*, *HIGH*), baseando-se em múltiplas variáveis difusas capturadas durante o *fuzzing*.

Objetivos Específicos

- Definir e implementar funções de pertinência para as três variáveis de entrada: desvio de *status* (*status_deviation*), tempo de resposta (*response_time*) e razão de *payload* (*payload_ratio*).
- Codificar **regras de inferência** do tipo SE-ENTÃO que traduzam o conhecimento heurístico sobre anomalias de software em decisões automatizadas.
- Integrar o **motor fuzzy** (*Fuzzy Engine*) ao analisador do *OpenAPI Contract Fuzzer* para processar resultados em tempo real.

- Validar a eficácia do sistema contra uma API vulnerável de referência (crAPI), documentando a distribuição e a precisão das severidades encontradas.

1.4. Justificativa e Motivação

A justificativa para a aplicação de Lógica Fuzzy neste projeto reside na natureza inerentemente “nebulosa” das métricas de desempenho e comportamento de servidores sob estresse. Métricas como tempo de resposta não possuem fronteiras rígidas; o que é considerado “lento” em um cenário de carga pode ser “aceitável” em outro. Ao fornecer um **score de criticidade contínuo** (0.0 a 1.0) em vez de um rótulo booleano, a ferramenta oferece informações muito mais ricas para que equipes de DevOps e segurança possam priorizar correções de forma eficiente.

Além disso, a solução é projetada para ser **leve**, permitindo sua execução em pipelines de Integração Contínua (CI/CD) sem adicionar um *overhead* computacional significativo. A escolha da biblioteca *scikit-fuzzy* para a implementação garante que o experimento seja não apenas reproduzível, mas também facilmente extensível para novos cenários de teste.

1.5. Delimitação do Escopo

Este trabalho delimita-se à **camada de inteligência** do processo de *fuzzing* — especificamente, o analisador fuzzy responsável por classificar as respostas obtidas. O *OpenAPI Contract Fuzzer*, responsável pela geração de *payloads* mutantes e execução das requisições, é utilizado como provedor de dados, não sendo alvo de modificações ou melhorias neste estudo. Não há mutação adaptativa realimentada pelo FIS (*feedback loop*) — o sistema é puramente classificatório. A validação experimental concentra-se em uma única API-alvo (crAPI) com aproximadamente 60 *endpoints*, o que limita a generalização dos resultados para outros contextos.

1.6. Estrutura do Trabalho

Este artigo está organizado em seis seções principais além desta introdução.

A Seção 2 apresenta a fundamentação teórica sobre Lógica Fuzzy, detalhando conceitos de conjuntos difusos, funções de pertinência e o modelo de inferência Mamdani.

A Seção 3 discute trabalhos relacionados, comparando esta abordagem com ferramentas de teste de API como *Schemathesis* e *Dredd*.

A Seção 4 detalha a metodologia, descrevendo a arquitetura do FIS, as variáveis de entrada e as 11 regras de inferência implementadas.

A Seção 5 apresenta e discute os resultados obtidos nos testes contra a *crAPI*, incluindo a distribuição de severidades e a análise de casos representativos.

Por fim, a Seção 6 traz as conclusões do estudo, respondendo à pergunta de pesquisa e sugerindo direções para trabalhos futuros.

2. Fundamentação Teórica

A **Lógica Fuzzy** (ou Lógica Difusa) foi introduzida por Lotfi A. Zadeh em 1965 como uma extensão da lógica clássica (booleana) para lidar com o conceito de verdade parcial.

Enquanto na lógica tradicional um elemento pertence ou não pertence a um conjunto (0 ou 1), a lógica fuzzy permite que elementos possuam **graus de pertinência** variando continuamente no intervalo unitário. Essa característica a torna ideal para modelar sistemas onde as fronteiras entre categorias são imprecisas ou “nebulosas”, como a distinção entre um tempo de resposta “normal” e “lento” em uma rede.

2.1. Conjuntos Difusos e Funções de Pertinência

Diferente da lógica binária, um **conjunto difuso** A em um universo de discurso X é definido por uma **função de pertinência** $\mu_A : X \rightarrow [0, 1]$, que mapeia cada elemento x a um valor que indica o quanto ele pertence àquele conjunto. As funções de pertinência podem assumir diversas formas geométricas dependendo da natureza do dado:

- **Função Triangular** (`trimf`): Definida por três parâmetros $[a, b, c]$, onde o valor de pertinência cresce linearmente de a até o pico em b , e decresce até c . É utilizada para representar valores com um ponto central de máxima precisão.
- **Função Trapezoidal** (`trapmf`): Definida por quatro parâmetros $[a, b, c, d]$, possuindo um platô de pertinência máxima entre os pontos b e c . É útil para representar intervalos de valores que são considerados plenamente membros do conjunto.

No projeto desenvolvido, estas funções são implementadas via biblioteca *scikit-fuzzy* [Warner et al. 2024] para caracterizar as variáveis de entrada. O Código 1 apresenta a definição completa das funções de pertinência e das regras de inferência implementadas na classe `FuzzyEngine`.

```
1 dev["low"] = fuzz.trimf(dev.universe, [0, 0, 0.2, 0.4])
2 dev["medium"] = fuzz.trimf(dev.universe, [0.2, 0.5, 0.8])
3 dev["high"] = fuzz.trapmf(dev.universe, [0.6, 0.8, 1, 1])
4
5 rtime["normal"] = fuzz.trapmf(rtime.universe, [0, 0, 200, 500])
6 rtime["slow"] = fuzz.trimf(rtime.universe, [200, 800, 2000])
7 rtime["timeout"] = fuzz.trapmf(rtime.universe, [1000, 2500, 5000,
8     5000])
9
10 psize["expected"] = fuzz.trapmf(psize.universe, [0, 0, 0.5, 1.5])
11 psize["large"] = fuzz.trimf(psize.universe, [1, 2, 3])
12 psize["empty"] = fuzz.trimf(psize.universe, [0, 0, 0.3])
13
14 ascore["low"] = fuzz.trapmf(ascor.universe, [0, 0, 0.2, 0.4])
15 ascore["medium"] = fuzz.trimf(ascor.universe, [0.25, 0.5, 0.75])
16 ascore["high"] = fuzz.trapmf(ascor.universe, [0.6, 0.8, 1, 1])
17
18 rules = [
19     ctrl.Rule(dev["high"] | rtime["timeout"], ascor["high"]),
20     ctrl.Rule(dev["medium"] & rtime["slow"], ascor["medium"]),
21     ctrl.Rule(dev["low"] & rtime["normal"]
22         & psize["expected"], ascor["low"]),
23     ctrl.Rule(dev["medium"] & rtime["normal"]
24         & psize["large"], ascor["medium"]),
25     ctrl.Rule(dev["low"] & psize["empty"], ascor["medium"]),
26     ctrl.Rule(dev["high"], ascor["high"]),
27     ctrl.Rule(psize["large"] & rtime["timeout"], ascor["high"]),
28     ctrl.Rule(dev["low"] & psize["large"], ascor["medium"]),
29     ctrl.Rule(dev["low"] & rtime["slow"], ascor["medium"]),
```

Listing 1. Funções de pertinência e regras do FIS (`fuzzy_engine.py`)

2.2. Fuzzificação

A **fuzzificação** é a etapa inicial do **Sistema de Inferência Fuzzy (FIS)**, na qual valores numéricos precisos (*crisp*) extraídos do mundo real — como um tempo de resposta de 300 ms — são convertidos em graus de pertinência nos conjuntos difusos definidos. Por exemplo, um tempo de resposta pode ser mapeado simultaneamente como “0.75 de pertinência no conjunto *normal*” e “0.25 de pertinência no conjunto *lento*”, permitindo que o sistema processe a transição gradual entre esses estados em vez de uma quebra abrupta.

2.3. Base de Regras e Inferência

O “cérebro” do sistema reside na **Base de Regras**, que consiste em um conjunto de sentenças linguísticas no formato **SE** (antecedente) **ENTÃO** (consequente). Estas regras modelam o conhecimento heurístico sobre a gravidade das falhas em APIs. O processo de inferência utiliza operadores lógicos para combinar os antecedentes:

- **Operador AND (E):** Geralmente calculado como o *mínimo* entre os graus de pertinência dos antecedentes.
- **Operador OR (OU):** Geralmente calculado como o *máximo* entre os graus de pertinência.

Para este trabalho, foram codificadas **11 regras de inferência**. As regras fundamentais incluem:

1. *SE* o desvio de *status* é **alto** *OU* o tempo de resposta é **timeout**, *ENTÃO* a criticidade é **alta**.
2. *SE* o desvio é **médio** *E* o tempo é **lento**, *ENTÃO* a criticidade é **média**.
3. *SE* o *payload* é **grande** *E* o *status* é documentado, *ENTÃO* a criticidade é **média** (indicando potencial vazamento de dados).

2.4. Desfuzzificação

A etapa final é a **desfuzzificação**, que converte o conjunto difuso resultante da combinação das regras de volta para um valor numérico único (*crisp*). O método utilizado nesta implementação é o **Centro de Massa (Centroid)**, que calcula o ponto de equilíbrio da área resultante das funções de pertinência ativadas.

O resultado é um **Score de Anomalia** variando de 0.0 a 1.0, que é então mapeado para categorias discretas de severidade para facilitar a interpretação pelo desenvolvedor:

- **LOW** (Baixa): $\text{Score} < 0.33$
- **MEDIUM** (Média): $0.33 \leq \text{Score} < 0.66$
- **HIGH** (Alta): $\text{Score} \geq 0.66$

3. Trabalhos Relacionados

A literatura sobre testes automatizados de APIs REST apresenta diversas ferramentas que utilizam contratos OpenAPI para guiar a geração de casos de teste. Entretanto, a maioria dessas soluções foca na **detecção binária** de falhas, sem oferecer uma gradação de severidade baseada em múltiplas variáveis como o sistema proposto.

Uma das ferramentas mais utilizadas na indústria é o **Schemathesis** [Kudriavtsev and contributors 2024], que emprega *fuzzing* baseado em propriedades (hipóteses) para gerar *payloads* que validam a conformidade dos *endpoints*. Embora seja extremamente eficaz na geração estatística de dados, o Schemathesis utiliza um oráculo de teste puramente booleano: a resposta é classificada como falha se violar o contrato ou retornar um erro 5xx, sem realizar um ranqueamento de criticidade entre diferentes anomalias detectadas.

De forma semelhante, o **Dredd** é uma ferramenta focada em validar se a implementação da API condiz com sua documentação. Ele executa requisições e compara os esquemas de resposta, mas sua análise limita-se à **validação estrutural** (*JSON Schema*), carecendo de heurísticas para identificar comportamentos anômalos de rede ou desempenho que não constituam uma quebra direta de contrato.

No campo acadêmico, [Arcuri et al. 2026] apresentam o **Tornado**, uma ferramenta voltada para a geração automática de oráculos de teste para APIs REST. O Tornado utiliza heurísticas para determinar o código de *status* esperado, mas, assim como as ferramentas industriais, baseia-se em decisões discretas.

Não foram encontrados trabalhos que integrem o *fuzzing* sintático-semântico à Lógica Fuzzy para tratar a incerteza e a gradação das respostas, o que caracteriza a originalidade desta proposta.

A aplicação de **Sistemas de Inferência Fuzzy (FIS)** em testes de software é geralmente voltada para a *priorização* de casos de teste em suítes de regressão, e não para a classificação automatizada do oráculo de execução em tempo real. Portanto, este trabalho preenche uma lacuna ao utilizar a inteligência difusa para transformar o contrato OpenAPI de um documento passivo em um **oráculo ativo** capaz de ranquear anomalias por sua relevância prática.

4. Metodologia

Esta seção detalha os métodos, ferramentas e etapas aplicados para a construção do oráculo de teste inteligente. A pesquisa caracteriza-se como aplicada e experimental, utilizando o *fuzzer* de contrato desenvolvido para coletar dados brutos que alimentam o motor de lógica fuzzy. O foco da metodologia reside na transformação de métricas de execução de APIs em indicadores de severidade quantificáveis.

4.1. Arquitetura do Sistema de Inferência Fuzzy (FIS)

O sistema foi projetado para atuar como a **camada de inteligência** (analisador) do projeto *OpenAPI Contract Fuzzer*. A arquitetura baseia-se em um modelo de inferência de **Mamdani**, implementado em Python com a biblioteca *scikit-fuzzy* [Warner et al. 2024]. O fluxo de dados consiste na captura de resultados de requisições (`results.json`),

seguida pela fuzzificação de três variáveis de entrada e, finalmente, a geração de um **score de anomalia** por meio de **11 regras de inferência**.

As variáveis que compõem o universo de discurso do FIS são apresentadas na Tabela 1.

Table 1. Variáveis de entrada do FIS

Variável	Universo	Conjuntos	Descrição
status_deviation	[0.0, 1.0]	low, medium, high	Desvio do código retornado vs. contrato
response_time	[0, 5000] ms	normal, slow, timeout	Tempo de latência da requisição
payload_ratio	[0.0, 3.0]	expected, large, empty	Proporção do tamanho do corpo da resposta

A variável de saída, `anomaly_score`, opera no intervalo [0.0, 1.0] e é mapeada para as categorias de severidade **LOW**, **MEDIUM** e **HIGH**.

4.2. Cálculo do Desvio de Status Code

O desvio de *status* é a variável primária para identificar falhas de conformidade de contrato. A lógica de cálculo penaliza códigos de erro não previstos na documentação OpenAPI.

- **Desvio 0.0:** Atribuído quando o código retornado está explicitamente documentado no contrato para o *endpoint* testado.
- **Desvio 0.6:** Atribuído a códigos da família 4xx não documentados. Representam comportamentos não especificados, mas que indicam que a API tratou a entrada, embora não tenha formalizado o erro.
- **Desvio 1.0:** Atribuído a erros internos do servidor (5xx) ou ausência de resposta (erros de rede). Estes casos indicam falhas críticas de robustez.

O Código 2 apresenta a implementação deste cálculo na classe `FuzzyEngine`.

```

1 def _compute_deviation(self, endpoint, result):
2     if result.status_code is None:
3         return 1.0 # sem resposta
4     actual = result.status_code
5     if 500 <= actual < 600:
6         return 1.0 # erro de servidor
7     expected = {int(s) for s in endpoint.responses if s.isdigit()}
8     if actual in expected:
9         return 0.0 # conforme contrato
10    if not expected:
11        return 0.5 # sem referencia
12    if 400 <= actual < 500:
13        return 0.6 # 4xx nao documentado
14    return 0.4 # outro desvio

```

Listing 2. Cálculo do desvio de *status* (`fuzzy_engine.py`)

4.3. Cálculo do Payload Ratio

A variável **Payload Ratio** avalia anomalias na estrutura dos dados retornados. Um desvio significativo no tamanho do corpo da resposta, especialmente em cenários de erro, pode

indicar **vazamentos de informações sensíveis** ou a exibição indevida de *logs* internos de depuração (*stack traces*).

O cálculo normaliza o tamanho da resposta em relação a uma linha de base esperada. Para aumentar a precisão da detecção de riscos, o peso desta variável é **dobrado** quando o código de *status* retornado também é um desvio do contrato, permitindo que as regras de inferência elevem o score de anomalia para o nível **HIGH** em casos de respostas volumosas não documentadas.

4.4. Geração de Payloads Mutantes

A eficácia do sistema de análise depende da qualidade dos dados gerados pelo motor de mutação. O *fuzzer* utiliza o contrato OpenAPI para criar um *payload* de referência (*baseline*) que respeita todos os tipos e restrições. A partir deste *baseline*, o motor de mutação (*engine.py*) aplica transformações em diferentes níveis:

- **Level 0:** Envio do *payload baseline* para validação de conformidade básica.
- **Level 1 (Mutações Unitárias):** Aplica uma única mutação por campo, focando em violações de tipo (ex: enviar *string* em campo numérico), extrapolação de limites (*minLength*, *maximum*) e injeção de caracteres especiais.
- **Level 2 (Mutações Combinadas):** Realiza múltiplas mutações em campos distintos simultaneamente, permitindo identificar falhas decorrentes da interação entre parâmetros.

O Código 3 ilustra as mutações aplicadas a campos dos tipos *string* e numérico.

```
1 class StringMutator(Mutator):
2     def mutate(self, path, schema):
3         return [
4             MutationOp(path, "replace", ""),           # string vazia
5             MutationOp(path, "replace", "A" * 500),    # overflow
6             MutationOp(path, "replace", 123),          # type confusion
7             MutationOp(path, "delete"),                 # omissao
8         ]
9
10 class NumericMutator(Mutator):
11     def mutate(self, path, schema):
12         return [
13             MutationOp(path, "replace", -1),           # negativo
14             MutationOp(path, "replace", 0),            # zero
15             MutationOp(path, "replace", 999999),       # overflow
16             MutationOp(path, "replace", "NaN"),        # type confusion
17             MutationOp(path, "delete"),                 # omissao
18         ]
```

Listing 3. Exemplos de mutadores (*string.py* e *numeric.py*)

4.5. Auto-Auth e Auth Probes

Para garantir a cobertura de *endpoints* protegidos, o sistema implementa o módulo **Auto-Auth**, que automatiza a descoberta de fluxos de registro e login no contrato, extraindo *tokens* JWT para injetar o cabeçalho *Authorization* nas requisições subsequentes. Complementarmente, são executadas **Auth Probes** antes de qualquer mutação. Para cada *endpoint* autenticado, o *fuzzer* envia duas requisições de controle:

1. Sem *token* de autenticação.
2. Com um *token* inválido (Authorization: Bearer invalid_token).

O resultado dessas *probes* é fundamental para o analisador classificar se a API possui um controle de acesso robusto ou se apresenta uma **contradição de erro** (ERROR_CONTRADICTION) ao não rejeitar acessos indevidos conforme o contrato.

4.6. Classificador Binário (Anomaly Types)

Antes da aplicação da inteligência difusa, cada resposta é submetida a um classificador binário discreto. Este módulo categoriza o resultado em tipos específicos definidos pelo enumerador do Código 4.

```
1 class Anomaly(Enum):
2     SERVER_ERROR # 5xx
3     SUCCESS_CONTRADICTION # 2xx com schema violado
4     UNEXPECTED_SUCCESS # 2xx para payload invalido
5     ERROR_CONTRADICTION # 4xx/5xx com schema violado
6     UNEXPECTED_ERROR_STATUS # status nao documentado
7     EXPECTED_FAILURE # rejeicao correta
```

Listing 4. Tipos de anomalia (classification.py)

O Sistema de Inferência Fuzzy (FIS) é aplicado **após** essa classificação inicial. Enquanto o classificador binário define a natureza do problema, o FIS atribui o score de severidade e o ranqueamento, permitindo que falhas de um mesmo tipo sejam priorizadas de acordo com sua “nebulosidade” (ex: um erro 500 lento vs. um erro 500 rápido).

4.7. Ambiente de Experimentação

Os experimentos foram conduzidos em uma estação de trabalho MacBook Air (16GB RAM) utilizando a linguagem Python 3.13. A API alvo escolhida foi a **crAPI** (*Completely Ridiculous API*) [OWASP Foundation 2024], um ambiente propositalmente vulnerável que expõe aproximadamente 60 *endpoints* distribuídos em domínios de identidade, mecânica e comunidade.

Para a execução de referência, foram adotados os seguintes parâmetros:

- **Nível de Fuzzing:** Level 1 (mutações unitárias).
- **Limites:** Máximo de 10 mutações por *endpoint*.
- **Políticas de Rede:** *Delay* de 100 ms entre requisições para evitar negação de serviço acidental e *timeout* de 15 segundos por chamada HTTP.

5. Resultados e Discussões

Nesta seção, são apresentados e discutidos os resultados obtidos na validação experimental do sistema. O foco da análise reside na capacidade do motor *fuzzy* em transformar um volume bruto de dados de execução em informações categorizadas por criticidade, permitindo uma interpretação mais clara da robustez da API testada.

5.1. Visão Geral dos Resultados

A execução de referência utilizada para esta análise (identificada pelo diretório de saída `output/20260626_105901`) submeteu a API `crAPI` [OWASP Foundation 2024] a um regime de testes de nível 1 (mutações unitárias), com um limite de até 10 mutações por *endpoint*. Ao longo do processo, o sistema identificou um total de **207 anomalias**, geradas a partir de requisições que provocaram algum nível de desvio em relação ao contrato OpenAPI [OpenAPI Initiative 2020] ou ao comportamento de rede esperado. Os dados foram processados pelo motor *fuzzy* em tempo real, resultando em um relatório consolidado que detalha tanto o tipo técnico da falha quanto o seu impacto estimado através do **Anomaly Score**.

5.2. Distribuição por Classificação das Anomalias

Antes da atribuição do score de severidade, as anomalias foram classificadas em tipos discretos de acordo com a natureza do erro. A Tabela 2 apresenta a distribuição dessas ocorrências.

Table 2. Distribuição das anomalias por classificação

Classificação	Quantidade	(%)
UNEXPECTED_ERROR_STATUS	103	49,8%
SERVER_ERROR	42	20,3%
UNEXPECTED_SUCCESS	28	13,5%
ERROR_CONTRADICTION	22	10,6%
EXPECTED_FAILURE	10	4,8%
SUCCESS_CONTRADICTION	2	1,0%

A análise qualitativa desses dados revela que a maioria das ocorrências (`UNEXPECTED_ERROR_STATUS`, 49,8%) refere-se a códigos de *status* 400 (*Bad Request*) que, embora corretos do ponto de vista semântico, não estavam documentados no contrato OpenAPI para os respectivos *endpoints*. As ocorrências de `SERVER_ERROR` (20,3%), por outro lado, representam o ponto mais crítico da análise. *Endpoints* como `/verify-email-token` e `/reset-password` falharam catastroficamente ao receber *payloads* mutantes, retornando códigos 500 ou 503. Em um cenário de análise booleana, todos os 207 achados seriam marcados apenas como “falha”, mas a classificação por tipo já começa a separar o que é ruído de documentação do que é uma quebra real de serviço.

5.3. Análise da Severidade via Lógica Fuzzy

A aplicação do Sistema de Inferência Fuzzy (FIS) permitiu refinar os 207 achados em três níveis de severidade, utilizando as funções de pertinência de saída baseadas no **Anomaly Score** calculado. A Tabela 3 apresenta a distribuição obtida.

Table 3. Distribuição das anomalias por severidade

Severidade	Intervalo	Quantidade
HIGH (Alta)	≥ 0.66	42
MEDIUM (Média)	$0.33 \leq score < 0.66$	120
LOW (Baixa)	< 0.33	45

O resultado demonstra o valor prático da inteligência difusa: apenas **20,3%** das anomalias detectadas foram classificadas como severidade alta. Esses achados concentraram as maiores pontuações devido à combinação de erros de servidor (5xx) e tempos de resposta elevados, disparando a regra de inferência de criticidade máxima.

As anomalias de severidade **MEDIUM** representam a maior parte dos resultados (58%) e consistem principalmente em erros 400 não documentados que não causaram lentidão nem geraram *payloads* excessivamente grandes. Ao isolar as 42 falhas de nível **HIGH**, o sistema permite que o desenvolvedor concentre seus esforços de correção no subconjunto de problemas que efetivamente compromete a disponibilidade ou a segurança da API, reduzindo a fadiga causada por falsos positivos ou problemas menores de documentação.

5.4. Análise de Casos

Para ilustrar o funcionamento do oráculo difuso, foram selecionados quatro casos representativos que demonstram como as variáveis de entrada influenciam o score final de criticidade.

Caso 1 — Erro de Servidor Crítico (score=0.84, HIGH) No *endpoint* `/verify-email-token`, o envio de uma *string* vazia no campo de *token* resultou em um erro **500 Internal Server Error**. O motor *fuzzy* calculou um desvio de *status* de 1.0 e identificou um tempo de resposta de aproximadamente 800 ms. A ativação da regra de inferência “SE desvio é **alto**, ENTÃO score é **alto**” resultou na severidade máxima, indicando uma falha de robustez onde uma entrada malformada causou uma exceção não tratada no servidor.

Caso 2 — Erro Não Documentado (score=0.6, MEDIUM) Durante o teste do *endpoint* de login, mutações unitárias provocaram erros 400 *Bad Request* que não constavam no contrato OpenAPI. Como a API rejeitou a requisição (comportamento seguro), mas o *status* não estava documentado, o sistema atribuiu um desvio médio de 0.6. Com latência baixa (~50 ms) e *payload* pequeno, o FIS classificou o achado como severidade média, sinalizando uma necessidade de atualização da documentação (*Contract Drift*) em vez de uma falha de segurança imediata.

Caso 3 — Sucesso Inesperado (score=0.5, MEDIUM) No caso do *endpoint* `/reset-password`, a API aceitou uma senha excessivamente longa e campos vazios retornando *status* 200 OK. Embora o desvio de *status* tenha sido 0.0 (pois o *status* 200 é documentado), a variável `payload_ratio` atingiu um valor de ~1.8 devido ao eco do *payload* gigante na resposta. A regra “SE desvio é **baixo** E *payload* é **grande**, ENTÃO score é **médio**” foi ativada, alertando para uma falha de validação de entrada que um classificador booleano ignoraria por ler apenas o código de *status*.

Caso 4 — Contradição de Erro (score=0.31, LOW) Neste cenário, a API retornou um erro 400, mas o corpo da resposta omitiu campos obrigatórios definidos no esquema do contrato. Apesar de ser uma violação contratual, o tempo de resposta rápido e o *payload*

reduzido resultaram em um score baixo. O FIS interpretou que, embora haja uma falha de conformidade estrutural, a API cumpriu sua função primordial de rejeitar a entrada inválida.

5.5. Discussão — Eficácia do FIS

A análise dos 207 achados confirma que o Sistema de Inferência Fuzzy (FIS) cumpre o objetivo de prover granularidade ao oráculo de teste. Os principais acertos observados foram a correlação perfeita de scores altos (≥ 0.84) com falhas catastróficas (5xx) e a capacidade de filtrar falsos positivos de baixa prioridade através de scores baixos (0.17 a 0.36).

Entretanto, foram identificadas limitações:

- **Uniformidade na crAPI:** Devido à ausência generalizada de documentação de erros 400 na API alvo, muitas anomalias ficaram retidas no score médio (0.5 a 0.6). O sistema teve dificuldade em distinguir entre um “erro 400 inofensivo” e um “erro 400 com vazamento de dados” quando o tamanho do *payload* era similar.
- **Thresholds:** O limiar de 0.33 para a categoria **MEDIUM** mostrou-se pouco seletivo, concentrando 58% das anomalias nesta faixa. Ajustes futuros nas funções de pertinência para um *threshold* de 0.4 poderiam aumentar a seletividade da ferramenta.
- **Natureza Estática:** O FIS atual não possui um *loop* de retroalimentação; as regras são fixas e não se adaptam conforme o desenvolvedor confirma ou rejeita um falso positivo.

Apesar dessas limitações, o sistema demonstrou ser superior a uma abordagem binária ao permitir que o desenvolvedor concentre sua atenção nos **20,3%** de falhas críticas detectadas, em vez de ser sobrecarregado por uma lista exaustiva de desvios contratuais menores.

6. Conclusão

Este trabalho apresentou o desenvolvimento e a avaliação de um Sistema de Inferência Fuzzy (FIS) integrado a uma ferramenta de *fuzzing* guiado por contrato para a classificação inteligente de anomalias em APIs REST. A abordagem visou superar as limitações dos oráculos de teste binários, que frequentemente falham em capturar a nuance e a incerteza de comportamentos anômalos em ambientes de rede complexos.

6.1. Resposta à Pergunta de Pesquisa

Os resultados experimentais confirmam a hipótese de pesquisa: o uso de **Lógica Fuzzy** permite classificar anomalias de robustez com uma granularidade significativamente superior às abordagens booleanas tradicionais. Enquanto um classificador binário rotularia todos os 207 desvios detectados na crAPI [OWASP Foundation 2024] como “falhas” genéricas, o motor *fuzzy* foi capaz de isolar apenas **42 ocorrências (20,3%)** como de alta severidade, permitindo uma priorização imediata de erros críticos de servidor (5xx) e latências extremas.

A combinação das três variáveis — desvio de *status*, tempo de resposta e razão de *payload* — provou ser eficaz para capturar contextos que variáveis isoladas ignorariam, como falhas de validação de entrada que retornam *status* de sucesso, mas com respostas volumosas e lentas.

6.2. Contribuições

As principais contribuições deste estudo incluem:

- A implementação de um dos **primeiros Sistemas de Inferência Fuzzy** aplicados especificamente como oráculo de teste para *fuzzing* de APIs REST.
- A definição de um conjunto de **11 regras de inferência** baseadas em heurísticas de engenharia de software para a tradução de comportamentos de rede em scores de risco.
- A disponibilização de uma ferramenta *open-source* integrada ao *OpenAPI Contract Fuzzer*, facilitando a reprodutibilidade por outros pesquisadores.
- A validação empírica detalhada contra a API crAPI, gerando uma base de dados de **207 anomalias** classificadas e documentadas.

6.3. Trabalhos Futuros

Para a evolução desta pesquisa, propõem-se as seguintes frentes:

1. **Loop Adaptativo com Feedback Fuzzy:** Implementar um sistema de aprendizado onde a confirmação manual de um falso positivo pelo desenvolvedor ajuste automaticamente as funções de pertinência do FIS.
2. **Validação em Larga Escala:** Testar o motor *fuzzy* contra APIs de produção (como GitHub ou Stripe) para avaliar a generalização das regras de inferência em diferentes arquiteturas.
3. **Extração de Novas Features:** Incorporar variáveis adicionais como a presença de *stack traces* no corpo da resposta e a contagem de campos retornados *versus* esperados.
4. **Integração CI/CD Nativa:** Desenvolver um *plugin* de automação que utilize o score *fuzzy* para decidir, de forma autônoma, se um *build* deve ser interrompido (ex: apenas para scores > 0.8).

6.4. Considerações Finais

A **Lógica Fuzzy** demonstrou ser um arcabouço prático, leve e matematicamente rigoroso para lidar com a “nebulosidade” inerente ao teste de software moderno. Ao transformar o contrato OpenAPI de um documento estático em um **oráculo ativo e inteligente**, este trabalho contribui para o aumento da robustez de sistemas distribuídos e para a redução do esforço humano na triagem de falhas de software.

Referências

- Arcuri, A. et al. (2026). Automated test oracle generation for rest apis. *IEEE Transactions on Software Engineering*. Preprint/Early Access.
- Kudriavtsev, D. and contributors (2024). Schemathesis: Property-based testing for api schemas.
- Meyer, B. (1992). Applying ”design by contract”. *Computer*, 25(10):40–51.
- OpenAPI Initiative (2020). Openapi specification (v3.0.3).
- OWASP Foundation (2024). crapi: Completely ridiculous API (vulnerability playground). Accessed: 2026.
- Warner, J. et al. (2024). scikit-fuzzy: Fuzzy logic for python.
- Zadeh, L. A. (1965). Fuzzy sets. *Information and Control*, 8(3):338–353.